B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution, Affiliated to VTU, Belagavi)

Bull Temple Road, Basavanagudi, Bengaluru – 560 019

Alternative Assessment Tool -2(AAT-2) Report on

"Design and Simulation of a Universal Shift Register and a Divide-by-N Counter using Verilog HDL in Cadence"

Submitted in partial fulfilment of the requirements of the Alternative Assessment Tool (AAT) for the course

ADVANCED DIGITAL AND LOGIC DESIGN (ADLD)

[Course Code: 23ECSPE1AD]

Submitted by

Harshitha U S – 1BM24EC072

Submitted to

Prof. Ajaykumar Devarapalli

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

BMS College of Engineering

Bull Temple Road, Basavanagudi, Bengaluru-560019

Academic Year

2026 – 2027

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to **Prof. Ajaykumar Devarapalli** for his valuable guidance, encouragement, and support throughout the completion of this Alternative Assessment Tool - 2(AAT-2) on "Design and Simulation of a Universal Shift Register and a Divide-by-N Counter using Verilog HDL in Cadence"

We extend our thanks to the **Department of Electronics and Communication Engineering, BMS College of Engineering**, for providing the necessary facilities and resources to carry out this work.

We are also thankful to our faculty members and everyone who directly or indirectly supported us in completing this report successfully.

Finally, we thank our team members for their cooperation and contribution towards the successful completion of this AAT.

INDEX

Sl No.	Contents	Page No.
1.	1. Abstract	1
	2. Software and Tools	1
2.	Experiments	
	1. Universal Shift Register (4-bit)	2-6
	2. Divide-by-N Counter	7-10

1.Abstract

This report presents the design and simulation of two sequential digital modules, a 4-bit universal shift register and a parameterized divide-by-N counter, using Verilog HDL and the Cadence simulation environment (Xcelium simulator with the SimVision waveform viewer). These two modules were first introduced in AAT-1 using Xilinx Vivado. In AAT-2 they are re-implemented and verified in Cadence, and the shift register is extended from a simple serial-in/serial-out register to a universal register supporting hold, shift right, shift left and parallel load. Each experiment includes the aim, design description, Verilog source code, testbench, simulation results with waveform analysis, and reserved space for the circuit/RTL schematic and waveform screenshots. Simulation videos and all source files are hosted in the GitHub repository listed at the end of this report.

2. Software and Tools

Software: Cadence Xcelium Logic Simulator with SimVision waveform viewer (optionally Cadence Genus for RTL schematic generation).

Language: Verilog HDL.

Simulation is performed using testbenches that apply input sequences, while the output is observed in the SimVision waveform window and the terminal log.

General Procedure in Cadence

- Create a working directory and place the design file (.v) and its testbench (.v) in it.
- Open a terminal and load the Cadence environment on the lab machine.
- Compile, elaborate and simulate with Xcelium, for example: xrun -access +rwc design.v tb.v -gui
- The testbench contains $shm_open and $shm_probe("AS") calls so that all signals are recorded in a waveform database.
- In SimVision, select the testbench signals in the design browser and send them to the waveform window.
- Use *Zoom Fit* and place cursors to read signal values at required times.
- Capture the schematic (SimVision Schematic Tracer, or Genus schematic view) and the waveform, then paste the screenshots in the spaces provided in each experiment.
- Record a screen video of the simulation run for each circuit.

2. EXPERIMENTS

1. Universal Shift Register (4-bit)

Aim

To design and simulate a 4-bit universal shift register that can hold its data, shift right, shift left, and load data in parallel, selected by a 2-bit mode control input.

Design Description

Inputs: clk, rst (active high, synchronous), s[1:0] (mode select), d[3:0] (parallel data), lsb_in (serial input used while shifting left), msb_in (serial input used while shifting right). Output: q[3:0].

On every rising edge of clk, the register behaves according to the mode select s[1:0]. Structurally, each flip-flop has a 4:1 multiplexer at its D input that chooses between its own output (hold), the neighbour on its left (shift right), the neighbour on its right (shift left) and the parallel input bit (load).

Mode control table:

s[1]	s[0]	Operation	Next state q[3:0]
0	0	Hold	q[3:0]
0	1	Shift right	{msb_in, q[3:1]}
1	0	Shift left	{q[2:0], lsb_in}
1	1	Parallel load	d[3:0]

Verilog HDL Design Code

```
module shift_register_univ (
   input  wire       clk, rst,
   input  wire [1:0] s,
   input  wire [3:0] d,
   input  wire       lsb_in,   // enters LSB on shift left
   input  wire       msb_in,   // enters MSB on shift right
   output reg  [3:0] q
```

```
);
always @(posedge clk) begin
   if (rst)
      q <= 4'b0000;
   else begin
      case (s)
         2'b00: q <= q;                // hold
         2'b01: q <= {msb_in, q[3:1]};   // shift right
         2'b10: q <= {q[2:0], lsb_in};   // shift left
         2'b11: q <= d;                // parallel load
      endcase
   end
end
endmodule
```

Testbench (TB)

```
`timescale 1ns/1ps
module tb_shift_register_univ;
reg clk = 0, rst = 1, lsb_in = 0, msb_in = 0;
reg [1:0] s = 2'b00;
reg [3:0] d = 4'h0;
wire [3:0] q;

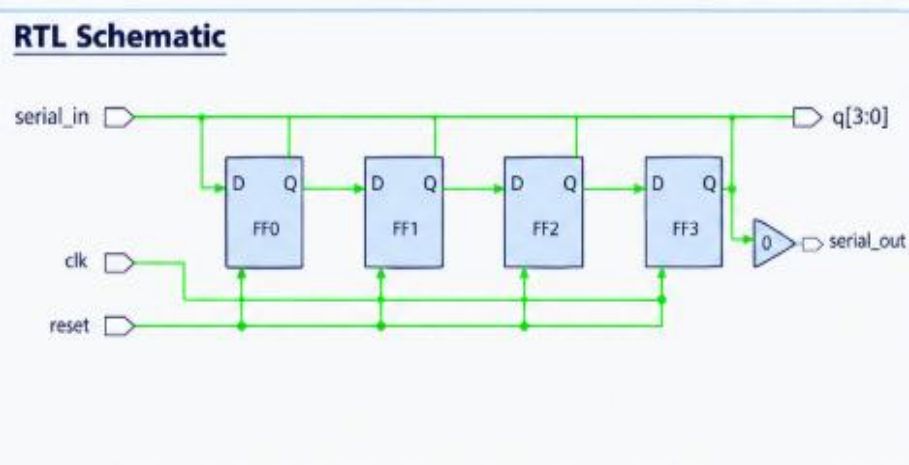
shift_register_univ dut (.clk(clk), .rst(rst), .s(s), .d(d),
                         .lsb_in(lsb_in), .msb_in(msb_in), .q(q));

always #5 clk = ~clk;
initial begin
```

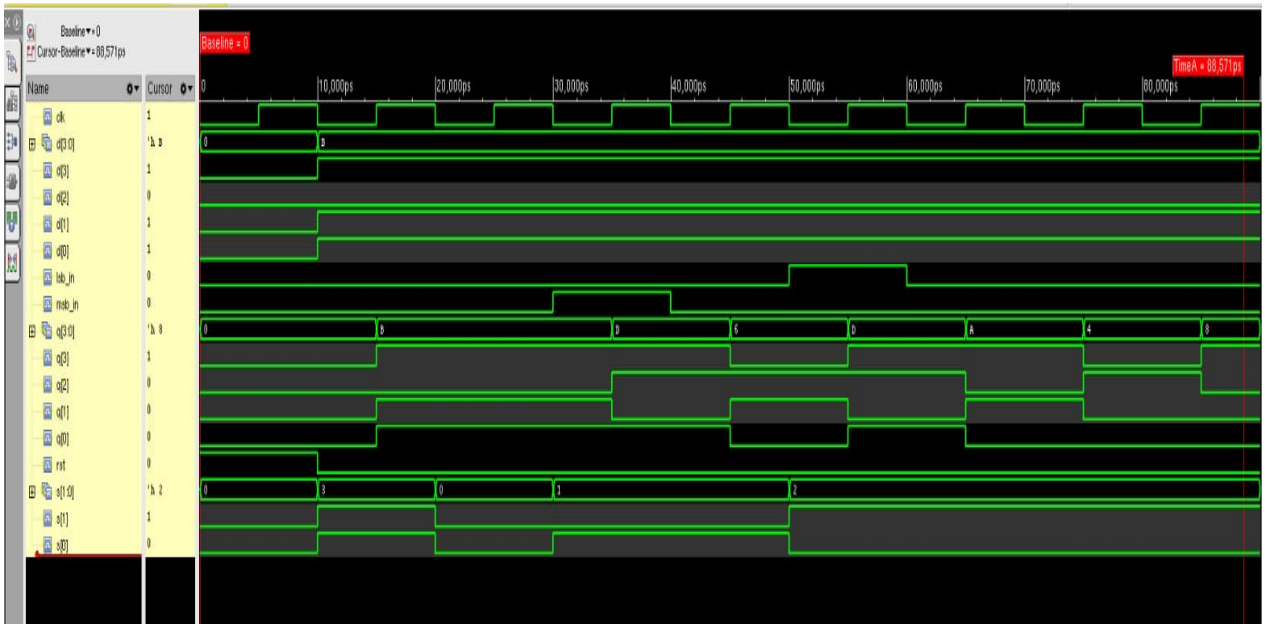
```
    $shm_open("shift_reg.shm");
    $shm_probe("AS");
    $monitor("t=%0t rst=%b s=%b d=%h lsb=%b msb=%b q=%h",
            $time, rst, s, d, lsb_in, msb_in, q);

    #10 rst = 0; d = 4'hB; s = 2'b11;   // 10 ns: parallel load B
    #10 s = 2'b00;                      // 20 ns: hold
    #10 s = 2'b01; msb_in = 1;          // 30 ns: shift right, msb_in = 1
    #10 msb_in = 0;                     // 40 ns: msb_in = 0
    #10 s = 2'b10; lsb_in = 1;          // 50 ns: shift left, lsb_in = 1
    #10 lsb_in = 0;                     // 60 ns: lsb_in = 0
    #40 $finish;                        // end at 100 ns
end
endmodule
```

Circuit / RTL Schematic



Simulation Waveform / Output



Waveform Analysis

The clock has a period of 10 ns with rising edges at 5, 15, 25 ns and so on. Reset is released at 10 ns. The table below lists the register contents after each rising edge.

Rising edge (ns)	s[1:0]	Inputs	Operation	q[3:0]
5	00	rst = 1	Reset	0000 (0)
15	11	d = 1011	Parallel load	1011 (B)
25	00	–	Hold	1011 (B)
35	01	msb_in = 1	Shift right	1101 (D)
45	01	msb_in = 0	Shift right	0110 (6)
55	10	lsb_in = 1	Shift left	1101 (D)
65	10	lsb_in = 0	Shift left	1010 (A)
75	10	lsb_in = 0	Shift left	0100 (4)
85	10	lsb_in = 0	Shift left	1000 (8)
95	10	lsb_in = 0	Shift left	0000 (0)

Result

The universal shift register correctly performs reset, parallel load, hold, shift right and shift left. The simulated values of q[3:0] match the expected values in the table above.

2. Divide-by-N Counter

Aim

To design and simulate a parameterized clock divider that toggles its output after a programmable number of input clock cycles, and to verify it for N = 5.

Design Description

Inputs: clk, rst (active high, synchronous). Output: clk_out. Parameter N is the number of input rising edges between two toggles of the output. The counter width is derived from N as CW = $clog2(N), which gives a 3-bit counter count[2:0] for N = 5.

The counter increments from 0 up to N−1. When it reaches N−1, it returns to 0 and clk_out toggles. A full output period therefore contains 2N input clock cycles, so the output frequency is f_out = f_in / (2N). For N = 5 the circuit is a divide-by-10 clock divider.

Control table:

rst	Counter condition	Behavior
1	X	count = 0, clk_out = 0
0	count < N−1	Increment count
0	count = N−1	Clear count and toggle clk_out

Verilog HDL Design Code

```
module divide_by_n #(
   parameter N = 5
) (
   input  wire clk, rst,
   output reg  clk_out
);
localparam CW = (N <= 1) ? 1 : $clog2(N);
reg [CW-1:0] count;

always @(posedge clk) begin
   if (rst) begin
```

```
        count   <= 0;
        clk_out <= 0;
    end else if (count == N-1) begin
        count   <= 0;
        clk_out <= ~clk_out;
    end else begin
        count   <= count + 1'b1;
    end
end
endmodule
```

Testbench (TB)

```
`timescale 1ns/1ps
module tb_divide_by_n;
parameter N = 5;
reg clk = 0, rst = 1;
wire clk_out;

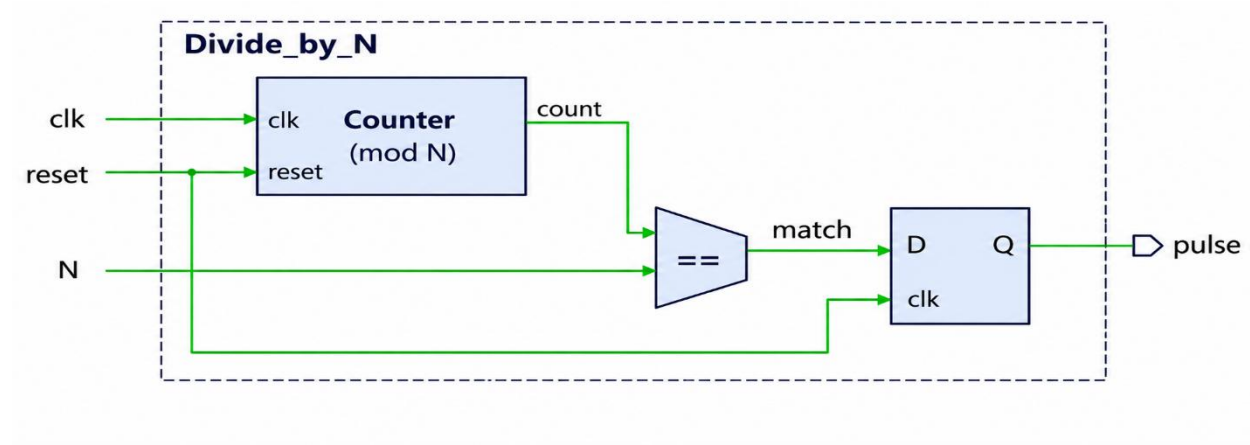
divide_by_n #(.N(N)) dut (.clk(clk), .rst(rst), .clk_out(clk_out));

always #5 clk = ~clk;

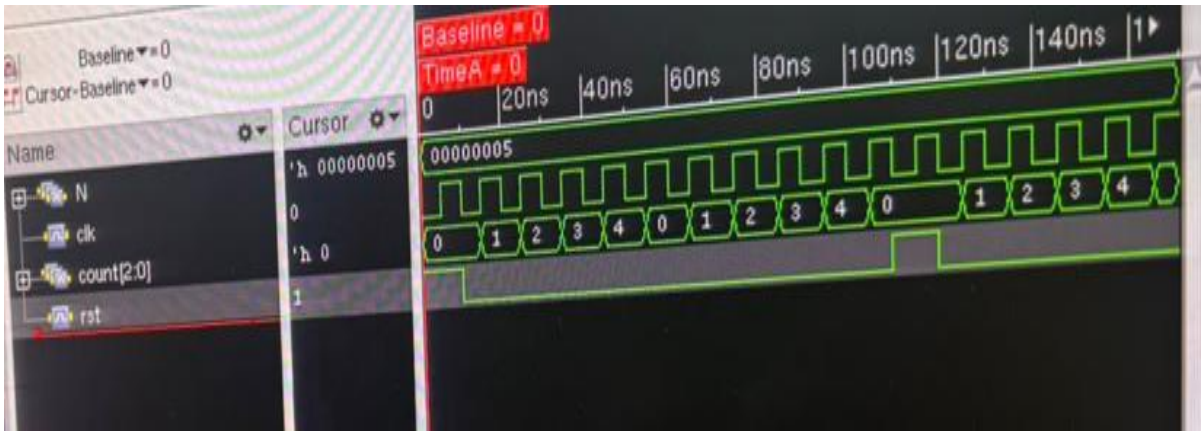
initial begin
    $shm_open("div_n.shm");
    $shm_probe("AS");
    #12  rst = 0;       // release reset
    #100 rst = 1;       // second reset pulse (about 110 ns)
    #10  rst = 0;
```

```
    #100 $finish;
end
endmodule
```

Circuit / RTL Schematic



Simulation Waveform / Output



Waveform Analysis

- Parameter N is displayed as the 32-bit value 00000005 in the waveform.
- While rst is high, count is held at 0 and clk_out is 0.
- After reset is released, count follows the sequence 0 → 1 → 2 → 3 → 4 → 0 on successive rising clock edges. The count repeats every N = 5 clock cycles.
- Each time count returns from 4 to 0, clk_out toggles. The output period is therefore 10 input clock cycles (100 ns for a 10 ns clock), which is divide-by-10.
- The second reset pulse near 110 ns clears count to 0 and clk_out to 0 immediately, after which the sequence restarts from 0.

Parameter	Value
Input clock period	10 ns
N	5
Counter width	3 bits
Output toggles every	5 input clock cycles (50 ns)
Output period	100 ns
Output frequency	f_in / 10

Result

The divide-by-N counter correctly divides the input clock by 2N = 10 for N = 5 and responds correctly to reset.

Conclusion

A 4-bit universal shift register and a parameterized divide-by-N counter were described in Verilog HDL and verified with testbenches in the Cadence Xcelium and SimVision environment. The shift register produced the expected values for all four modes, and the divide-by-N counter produced a clock output at one-tenth of the input frequency for N = 5. The simulation waveforms, schematics, source code and simulation videos are available in the project repository.

GitHub Repository

All reports, source code, waveforms and simulation videos for AAT-1 and AAT-2:

https://github.com/Harshitha-US/ADLD_AAT/tree/main

References

1. Cadence Design Systems, Xcelium Logic Simulator User Guide.
2. Cadence Design Systems, SimVision Waveform Viewer User Guide.
3. Xilinx, Vivado Design Suite User Guide: Logic Simulation (UG900).
4. Course notes and laboratory instructions for Verilog HDL and digital system design.